



Dart



Pemrograman Mobile Lanjut

Pertemuan 5

OLEH : NANDANG HERMANTO

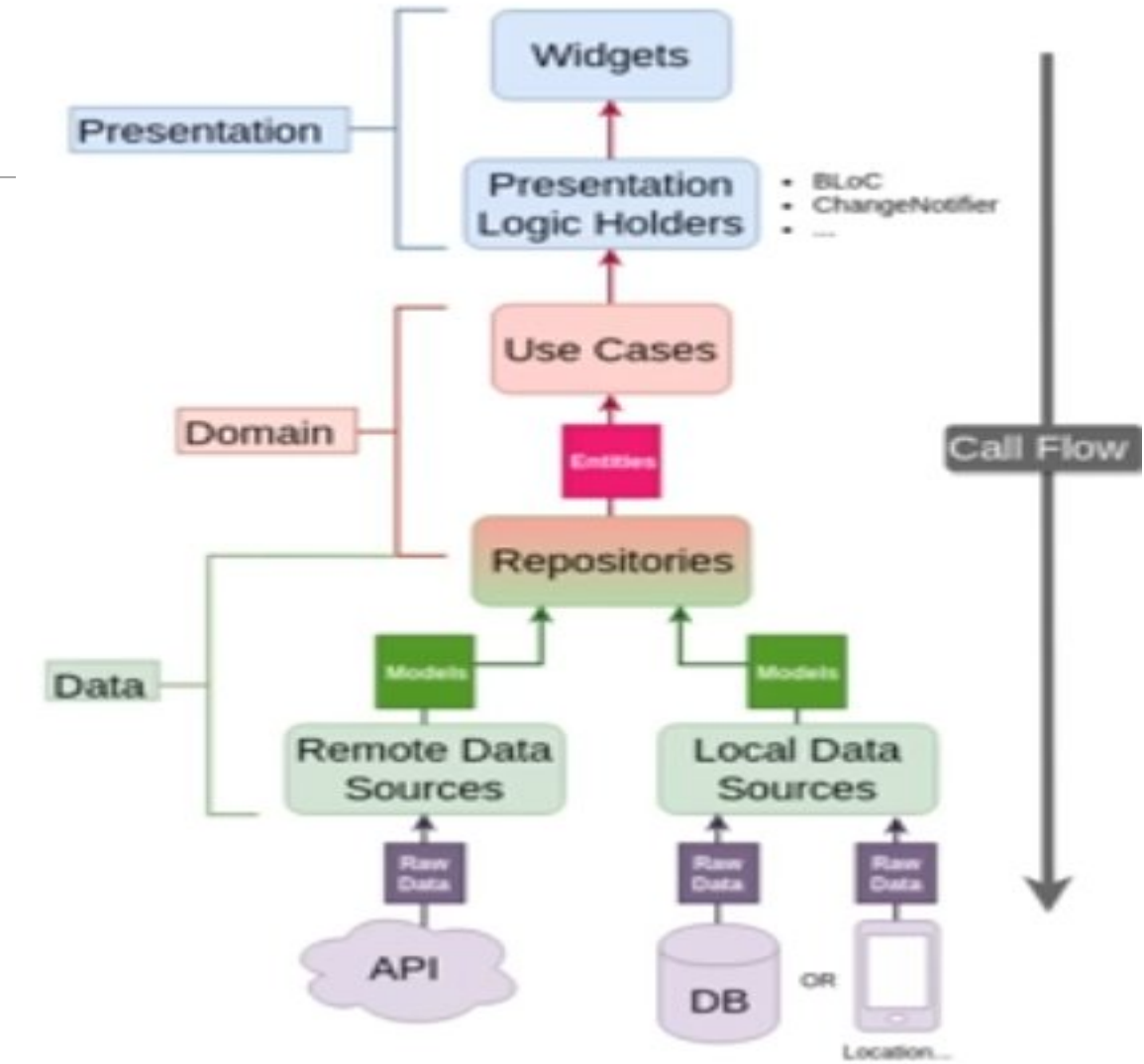
Clean Architecture dalam Flutter

Konsep Dasar Clean Architecture

1. Clean Architecture memisahkan aplikasi Flutter menjadi beberapa layer yang mandiri.
2. Fokus utamanya adalah separation of concerns (pemisahan tanggung jawab).
3. Dependency harus selalu mengarah ke dalam (dependency rule).
4. Business logic harus independen dari framework, database, dan UI.

Lapisan Utama Clean Architecture

1. Presentation Layer
2. Domain Layer
3. Data Layer

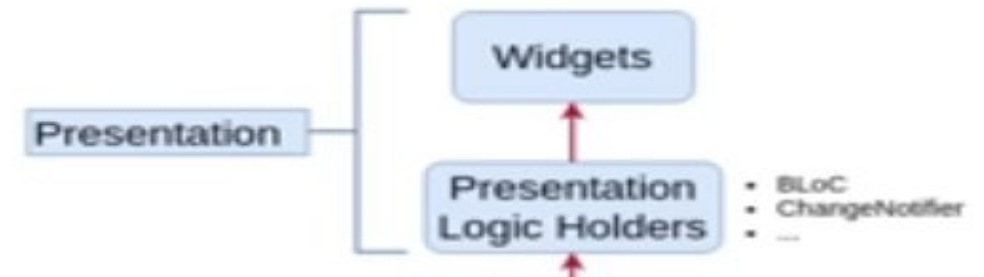


Presentation Layer

Lapisan terluar yang menangani antarmuka pengguna (UI) dan interaksi pengguna.

Komponen utama:

- Widget: membangun tampilan visual aplikasi.
- State Management: menggunakan BLoC, Riverpod, atau Provider.
- Presentation Logic Holder: kelas seperti BLoC atau ViewModel yang memicu Use Case.

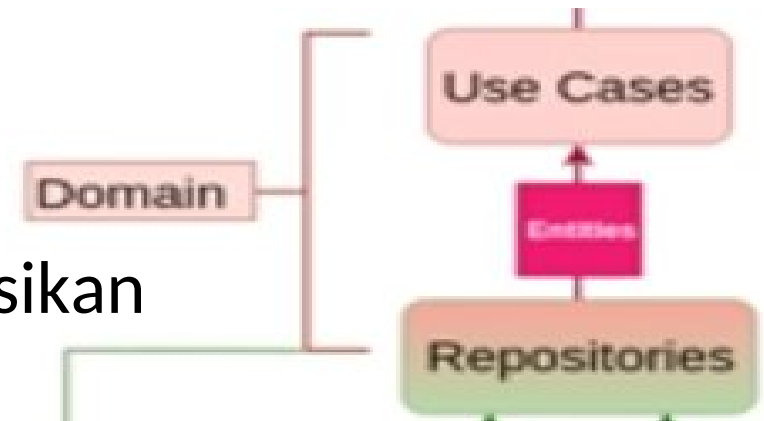


Domain Layer

Lapisan inti aplikasi yang berisi logika dan aturan bisnis (business rules).

Komponen utama:

- Entities: struktur data inti yang merepresentasikan objek bisnis.
- Repository Interfaces: kontrak pengambilan dan penyimpanan data.
- Use Cases / Interactors: logika bisnis untuk fitur tertentu.

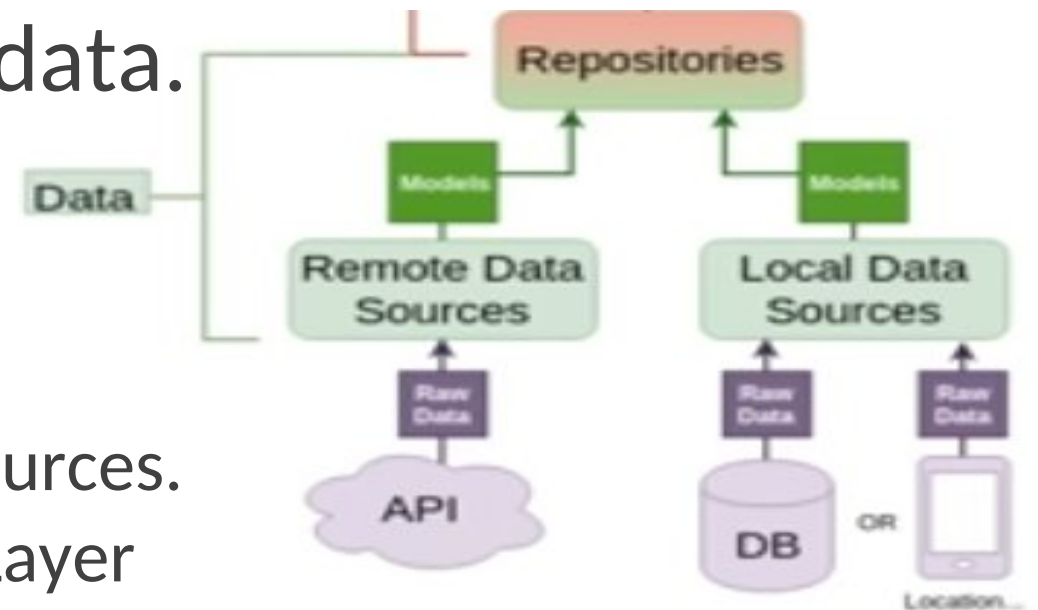


Data Layer

Bertanggung jawab atas pengambilan, penyimpanan, dan transformasi data.

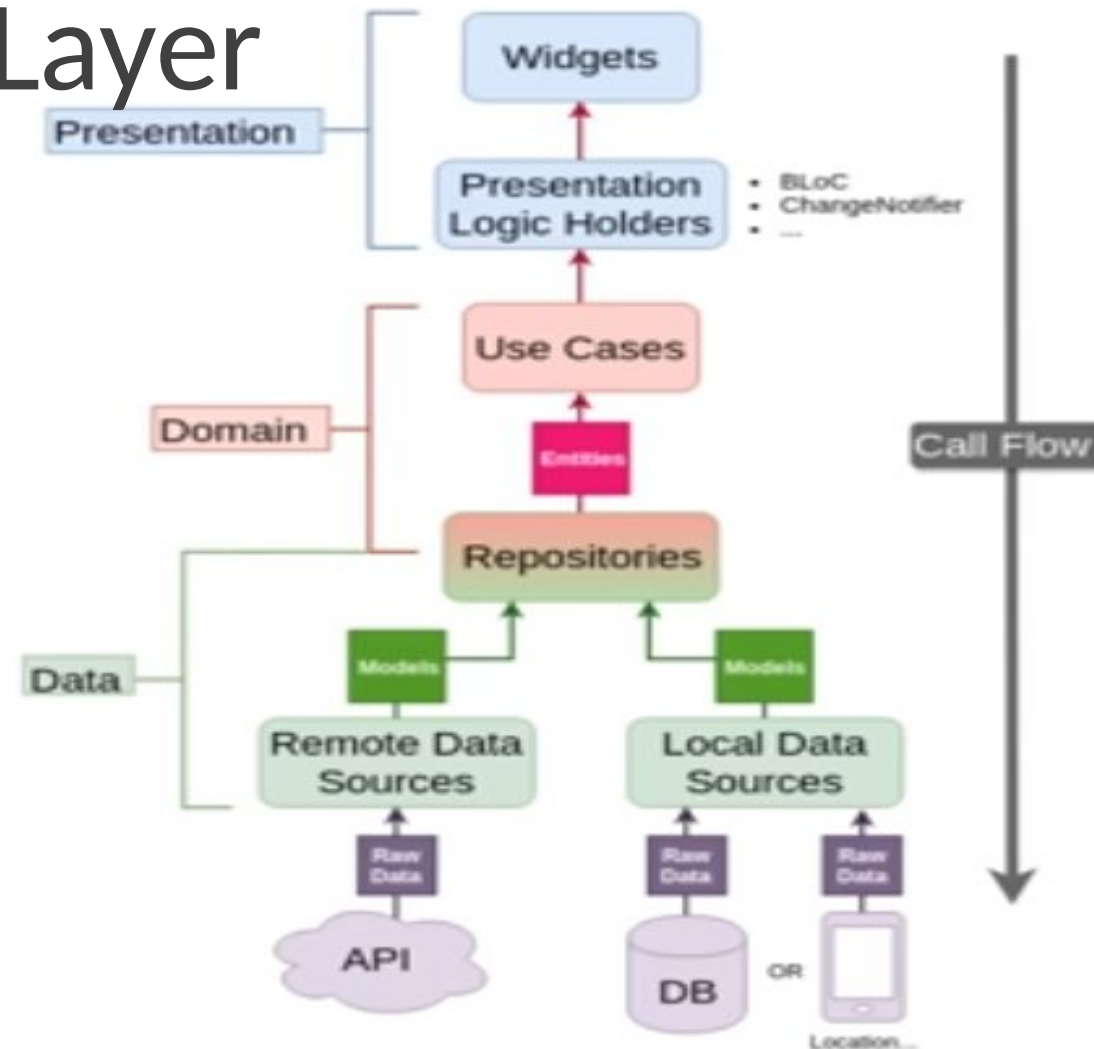
Komponen utama:

- Repositories: implementasi konkret dari Repository Interface.
- Data Sources: Remote dan Local data sources.
- Models: mentransfer data antara Data Layer dan sumber eksternal.



Alur Komunikasi Antar Layer

1. Widget memicu event di State Management.
2. State Management memanggil Use Case di Domain Layer.
3. Use Case menjalankan logika bisnis dan memanggil Repository Interface.
4. Repository Implementation di Data Layer mengambil data dari Data Source.
5. Data dikembalikan ke Presentation Layer untuk memperbarui UI.



Manfaat Clean Architecture

1. Testability: mudah diuji secara terpisah.
2. Maintainability: kode lebih terstruktur dan mudah dikelola.
3. Scalability: mudah menambah fitur tanpa memengaruhi bagian lain.
4. Independence: business logic tetap independen dari framework eksternal.

Kapan Menggunakan Clean Architecture

1. Proyek skala besar dan kompleks dengan banyak pengembang.
2. Proyek jangka panjang yang memerlukan maintainability tinggi.
3. Tidak direkomendasikan untuk proyek kecil karena kompleksitasnya tinggi.

Kesimpulan

1. Clean Architecture membantu menjaga aplikasi tetap modular, terstruktur, dan mudah dikembangkan.
2. Meski butuh usaha awal yang lebih besar, manfaat jangka panjangnya sangat signifikan.
3. Gunakan pendekatan ini pada proyek Flutter berskala menengah hingga besar.

Referensi

Dokumentasi & buku:

1. Orlova (2024), Bab 3 Clean Architecture Principles
2. Vashisht (2023), Bab 2 Architectural Patterns in Flutter

Rekomendasi tambahan:

1. Uncle Bob - Clean Architecture (artikel & buku),
2. dokumentasi get_it

Terimakasih
